

QA Module - C-Zentrix Data Pipeline

Technical Documentation & Setup Guide

Version: 1.1.0 (Production Release)

Date: June 27, 2026

System Type: Asynchronous Microservice (Celery, Redis, Docker)

1. Executive Summary

This document outlines the architecture, configuration, and maintenance procedures for the **QA Module Data Script**. The service acts as an autonomous background daemon that polls audio recordings from the C-Zentrix Campaign Manager, processes them through the Deepgram Transcription API, and uploads the resulting transcripts to the TVT QA Module.

2. Core Architecture

The system is built for high-throughput and stability, utilizing the following stack: - **Python 3.10**: Core scripting language. - **Celery & Redis**: For distributed task queueing and background processing. - **Gevent**: Green threads used by Celery workers to handle thousands of concurrent API requests without blocking. - **MySQL**: Persistent tracking database to prevent duplicate processing. - **Docker**: Containerized deployment for consistent environments.

3. Production Enhancements (v1.1.0)

This version includes critical optimizations for enterprise-scale loads: - **Gevent Concurrency**: The worker runs on gevent with high concurrency limits, eliminating freezing during long transcription API polls. - **Database Connection Pooling**: Built-in thread-local storage reuses MySQL connections, preventing "Too many connections" failures. - **LRU Cache Protection**: A memory-leak safe LRUcache (10,000 files limit) replaces unbounded sets in the directory scanner. - **Resilient Network Handlers**: urllib3

retry adapters are attached to HTTP sessions to automatically recover from 502, 503, and 504 backend errors. - **Dynamic Transcription Strategy:** Supports dual processing modes: - url mode for lightweight JSON-based submission. - upload mode for multipart file streaming.

4. Configuration Reference (.env)

The .env file governs the behavior of the entire system. Key configurations include:

Variable	Description
PROCESS_MODE	Set to dir, db, or api to choose the data polling source.
TRANSCRIPTION_SUBMISSION_METHOD	url (JSON URL payloads) or upload (direct .wav upload).
TRANSCRIPTION_API_KEY	The secret X-API-Key to authenticate with the Transcription service.
RECORDING_BASE_URL	The domain prefix used to construct the final URL sent to the QA server (e.g., http://your-domain.com).
POLL_INTERVAL_SECONDS	How often the beat scheduler scans for new files (Default: 60s).
QA_API_BASE_URL	The endpoint for the QA module (e.g., https://your-domain.com/api/v1).

5. Task Flow & Data Lifecycle

1. **Discovery:** Celery Beat wakes up every 60 seconds and triggers poll_new_calls_task.
2. **Identification:** The scanner looks for new entries in MySQL or .wav files in the directory.
3. **Queueing:** Unique records are pushed to the Redis Celery Queue (DB 2).
4. **Transcription:** The worker requests transcription using the C-Zentrix Deepgram API and asynchronously polls for completion.
5. **QA Upload:** The script authenticates with the QA Module (handling token expiry automatically) and pushes the final JSON payload containing the recordingUrl and transcript.

6. Operational Guidelines

- **Starting the System:**

```
docker-compose up --build -d
```

- **Checking Logs:** Logs are heavily rotated and stored at `/var/log/czentrrix/qa_call_push.log`.
- **Database Reset:** If a file must be re-processed, manually delete its `session_id` from the `processed_sessions` table in the `czentrrix_campaign_manager` MySQL database.